Can Short Hypervectors Drive Feature-Rich GNNs? Strengthening the Graph Representation of Hyperdimensional Computing for Memory-efficient GNNs

Jihe Wang
School of Computer Science
Northwestern Polytechnical University
Xi'an, China
wangjihe@nwpu.edu.cn

Yuxi Han
Northwestern Polytechnical University
Engineering Research Center of Embedded System Integration, Ministry of Education
Xi'an, China
hyx956@mail.nwpu.edu.cn

Danghui Wang*
Northwestern Polytechnical University
Engineering Research Center of Embedded System Integration, Ministry of Education
Xi'an, China
wangdh@nwpu.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133067

***Abstract*—Hyperdimensional computing (HDC) based GNNs are significantly advancing the brain-like cognition in terms of mathematical rigorousness and computational tractability. However, the researches in this field seem to have a "*long vector consensus*" that the length of HDC-hypervectors must be designed to mimic that of cerebellar cortex, i.e., ten thousands of bits, to express human's feature-rich memory. To system architects, this choice presents a formidable challenge that the combination of numerous nodes and ultra-long hypervectors could create a new memory bottleneck that undermines the operational brevity from HDC. To overcome above problem, in this work, we shift our focus to rebuilding a set of more GNN-friendly HDC-operations, by which, short hypervectors are sufficient to encode rich features via enjoining the strong error tolerance of neural cognition. To achieve that, three behavioral incompatibilities of HDC with general GNNs, i.e., *feature distortion*, *structural bias*, and *central-node vacancy*, are found and successfully resolved for more efficient feature-extraction in graphs. Taken as a whole, a memory-efficient HDC-based GNN framework, called CiliaGraph, is designed to drive one-shot graph classifying tasks with only hundreds of bits in hypervector aggregation, which offers 1 to 2 orders of memory savings. The results show that, compared to the SOTA GNNs, CiliaGraph reduces the memory access and training latency by an average of 292× (up to 2341×) and 103× (up to 313×), respectively, while maintaining the competitively accuracy.**

***Index Terms*—GNN, hyperdimensional computing, memory, graph representation.**

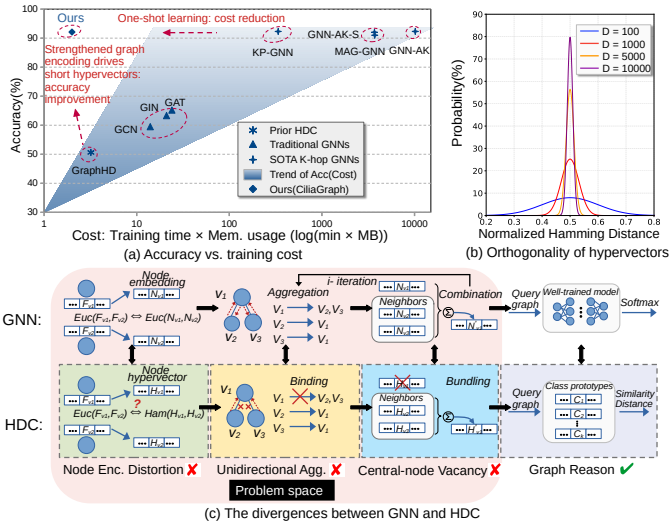

Fig. 1: Problem space of SOTA HDC and aims of our work.

I. Introduction

Hyperdimensional computing (HDC) utilizes long hypervectors to mimic the behaviors of the cerebellar cortex in brains, which successfully provides various desirable properties that other machine learning methods lack, such as simple operations, one-shot learning, and robustness to noise [1]. Accordingly, the brain-inspired HDC is introduced in traditional graph neural networks (GNNs) [2]–[5] to fully function its mathematical rigorousness [6] and computational tractability [7]. These HDC-based GNNs no longer require interminable training and expensive matrix operations, yet still achieve competitive model accuracy, as a result, being increasingly popular in graph cognitive tasks on edge devices [7]. Although the ultra-long hypervectors, e.g., tens of thousands bits, faithfully replicate the natural configuration in human brain, they pose a new challenge for system architects in terms of memory bottleneck, i.e., numerous nodes represented by long hypervector for each. Therefore, whether it is possible to encode rich features using shorter hypervectors remains an open question (Fig. 1(a)).

Hypervectors for rich features. Unlike NN-based feature encoding in general GNNs [8], HDC-based approaches commonly employ superposition of orthogonal hypervectors to fuse multiple features [9]. To forcibly convert numerical node features into hypervectors, two typical methods are widely adopted in their superposition: 1) ***random projecting*** [4], [5] and 2) ***leveled mapping*** [9]. The former directly scales the random hypervectors proportionally to their corresponding feature values and then sums them up to create a fused hypervector as the representation of multiple features, which always invokes a *majority function* [7] in final to revert the vectors to their binary forms, i.e., $(\{\mathbb{R}\}^D \rightarrow \{-1, 1\}^D$. The latter stochastically encodes feature values into pre-defined hypervectors as the weight to process "*bind-and-bundle*" operations (a.k.a. the "multiply-and-add" in HDC version) on corresponding orthogonal hypervectors, which removes the expensive *majority function*, meanwhile, retains the simple operation of binary. A common drawback of above methods is that features with clearly numerical implication become less distinguishable in hyper-dimensional spaces, particularly when a node has rich features, e.g., COIL-RAG [10] dataset even with 64 features. To compensate for the loss of feature representation, designers have been compelled to use a system-unfriendly choice that using a large amount of memory to hold much longer vectors [1], [4], [11], [12] for higher resolution of numerical features.

A key debate: Some fundamentalists of brain-inspire computing insisted on the necessity of ultra-long hypervectors for HDC-based GNNs. Their arguments come from a mathematical criterion that, in theory, hypervectors shorter than $10,000$-bits cannot ensure the high-quality of orthogonality (Fig. 1(b)), which has even been seen as the "*golden rule*" of any correct HDC [6], [11]. However, to system

*Corresponding author

architects, it's too expensive to use $10,000$-bits vectors to encode only one of node features. For example, to store one node with three features quantized to 8-bit FxP, its numerical embedding only needs 128 bytes in general GNNs [8], but unfortunately, the equivalent hypervector requires an unacceptable 29.3KB, which is almost out of reach at system level. Therefore, redundant hypervectors lead to severe inefficiency in memory, which casts doubt on the continued adherence of the "*golden rule*".

In this work, we find that the core difficulty of HDC-based GNNs is its inability to faithfully execute each steps involved in a standard GNN process [13], leading to three kinds of operational noises in below that significantly weaken the extraction of multiple features (Fig. 1(c)). ***Noise-1***: In feature encoding, by replacing the Euclidean distance with the simple Hamming one between hypervectors, both *random projecting* and *leveled mapping* introduce a distortion in the coding space, failing to accurately measure the relationships as the original numerical feature spaces. ***Noise-2***: The originally bidirectional and symmetric *aggregation* in GNNs has been simplified to a unidirectional tree-like *binding* in the existing HDC [3], [7], causing a structural bias of their graphs. ***Noise-3***: During *combination* stage, standard GNNs always fuse neighboring nodes with the central one (h^i) to generate a new embedding (h^{i+1}), however, existing HDCs, without exception, ignore central nodes via a socalled "equivalent" *bundling* [1], [4] which produces a *central-node vacancy* in the process of embedding fusion. As seen in Sec. II, these three divergences jointly result in the suppressed expressability of hypervectors, inevitably making low efficiency in memory usage.

To overcome the problems, we design a hypervector space that preserves the original numerical distribution of node features, meanwhile, enjoys the encoding efficiency of HDC. Based on the space, a fully symmetric on-edge *binding* is proposed to adhere the embedding aggregation in standard GNNs without the structural bias of graphs. To repair the vacated central nodes, a new concatenation operator is propose to merge periphery and central hypervectors before the final reason produced. Our proposed GNN framework, called CiliaGraph, demonstrates the feasibility of the aforementioned operations in one-shot graph classifying tasks using only 200-bit hypervectors. The results show that, compared with SOTA GNNs, CiliaGraph reduces memory usage and training time by 292× and 103× (up to 2341× and 313×) respectively. Our contributions include:

1. We identify the three sources of weak expression by hypervectors in HDC-based GNNs as the system-unfriendly compromise, i.e., long hypervectors, in the past.
2. CiliaGraph, including a set of new HDC operators, is proposed to mimics the information merging in GNNs, which demonstrates the feasibility of ultra short hypervectors in GNNs.
3. The results show the applicable of CiliaGraph in one-shot graph classifying tasks on widespread datasets and the great memory saving makes it well-suited for on-edge HDC-based GNNs.

II. Background and Motivation

A. HDC and the operators in GNNs

Traditional HDC models employ D-dimensional binary hypervectors as the fundamental elements for computation in $\mathcal{HD}$ space. Let's assume $\mathcal{H}_1, \mathcal{H}_2 \in \{-1, 1\}^D$ are two randomly generated hypervectors and their similarity $\delta(\mathcal{H}_1, \mathcal{H}_2) \approx 0$. ***Binding*** of $\mathcal{H}_1$ and $\mathcal{H}_2$, i.e., $\mathcal{H}_1 \otimes \mathcal{H}_2$, is component-wise multiplication (XOR in binary) and generates a new hypervector that is dissimilar to its two constituents, i.e., $\delta(\mathcal{H}_1 \otimes \mathcal{H}_2, \mathcal{H}_1) \approx 0$, which is used to find the association of two hypervectors [4]. ***Bundling*** is component-wise addition of two hypervectors, i.e., $\mathcal{H}_1 \oplus \mathcal{H}_2$, that memorizes its all

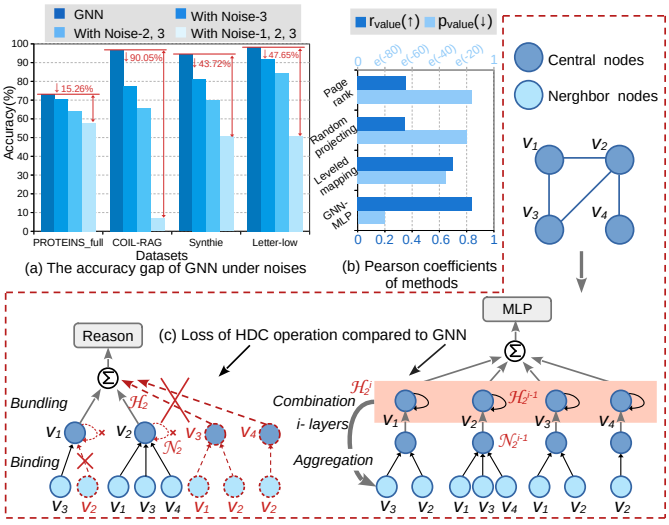

Fig. 2: A motivational analysis of accuracy loss under three noises introduced by HDC operators.

components, therefore, the result is similar to its every component, i.e., $\delta(\mathcal{H}_1 \oplus \mathcal{H}_2, \mathcal{H}_1) >> 0$. ***Reasoning*** is to measure the similarity of two hypervectors and generally utilizes Hamming distance or dot product [11]. The above three HDC primitives are adopted widely in the most HDC-based GNNs.

For a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, current HDC-based GNNs aim to mimic the feature extraction of traditional GNNs (Fig. 1(c)), where, 1) the node embeddings are recoded with ultra-long hypervectors, 2) the aggregation can be alternated as their binding, 3) the combination is just the bundling of all neighboring hypervectors, and 4) the final MLP is done efficiently with HDC reasoning. However, these purportedly "equivalent" operations exhibit unmatched capabilities in feature extraction on graphs. The ablation study (Fig. 2(a)) clearly demonstrates a stepwise decline, or noise, in the model inference accuracy as we progressively replace GNN operators with their HDC counterparts, i.e., combination→bundling (10.27% ↓), aggregation→binding (9.34% ↓), and embedding→hypervector (29.57% ↓) respectively. Resultantly, the SOTA HDC-based GNNs are generally 49.17% less accurate than traditional GNNs [3], [12], which has driven designers to the short-sighted solution of employing longer hypervectors, akin to a poisoned chalice. In next, the three deeper causes of these declines are analyzed.

B. Hypervector: Distortion of Distance in Feature Encoding

Effective numerical encoding using HDC has always been a challenging problem, primarily due to the inherent tension of ensuring both strict *orthogonality* and correct *numerical distances* between two hypervectors [6]. In practice, more efforts focus on maintaining the orthogonality, rather than the distances [1], [7], because they speculated that hypervectors, a.k.a. discrete random bit sequences, could represent the numerical disparities of features well as long as they are long enough. Unfortunately, our a comparative study on distance-isomorphism reveals a widespread severe distortion the hypervector spaces (Fig. 2(b)). This study summons Pearson's coefficient [14] to quantify **1)** r – the correlation between "the hypervector distances in $\mathcal{HD}$ spaces" and "the embedding distances of numerical features," and **2)** p – the error of r. The results shows that PageRank, Random-Projecting, and Leveled-Mapping, as the three most popular encoders in HDC ($r \downarrow, p \uparrow$), seriously deviate the expected GNN features ($r \uparrow, p \downarrow$), which confirms our doubt that long hypervectors are disable to encode the numerical features in GNNs. In this work, a plastic random bit-flip (PRBF) encoding is proposed

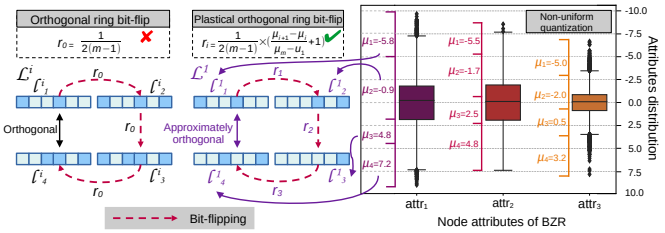

Fig. 3: Improved plastic orthogonal ring introduce a non-fixed flip rate compared to orthogonal ring.

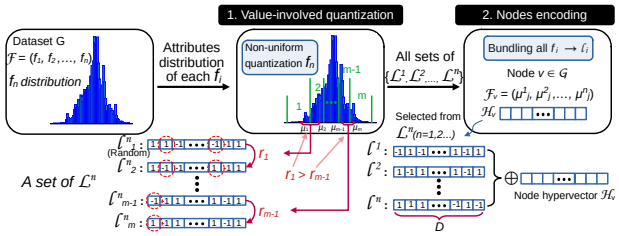

Fig. 4: The steps of plastic random bit-flip encoding

to fine-tune the random flips according to both feature values and their distributions, effectively overlaying numerical distance onto orthogonal hypervectors (Sec. III-A).

C. Binding: Structural Bias by Unidirectional Tree-like Aggregation

Standard GNNs are always capable of accurately learning graph structures, primarily by the bidirectional aggregation through edges, i.e., connectivity, which drives aggregation strictly adhering to the data-flow defined by a graph structure. In contrast, existing HDC-based one-shot solutions [3], which employ unidirectional binding to rapidly reduce long hypervectors to relieve memory pressure, simplify all complex graph structures into a tree that biases original graph structures. As shown in Fig. 2(c), while one edge in the original graph (e.g. $e_{1,2}$) triggers bidirectional binding in GNNs ($v_2 \rightarrow v_1$ for v_1 and $v_1 \rightarrow v_2$ for v_2), in HDC however, only the more important nodes (such as v_2) are responsible for collecting the neighbor features ($v_1 \rightarrow v_2$) and the reversed binding ($v_2 \rightarrow v_1$) are totally neglected. Reducing the symmetric GNN aggregations to half asymmetric ones limits the global structural information captured during the binding of HDC (Fig. 2(c)). In this work, a precise aggregation, i.e., *differential aggregation*, is proposed (Sec. III-B), which not only implements bidirectional symmetric aggregation as in GNNs but also leverages on-edge weights to modulate the strength of peripheral hypervectors, thereby achieving the full function of aggregating capabilities of GNNs.

D. Bundling: Vacancy of Central Nodes during Combination

The last drawback of HDC-based GNN lies in its perfunctory way in bundling, which abruptly overwhelms [4], [5], or even removes [3], the central nodes during the combination, resulting in a significant departure from the standard procedure of GNNs. As shown in Fig. 2(c), during the standard combination of GNNs, a node (e.g., v_2) stores a local embedding ($\mathcal{H}_2^{i-1}$) to memorize the embeddings of historical levels. By combining local embeddings with the aggregated peripheral ones of current layer, GNNs facilitate an embedding update of central node, e.g., $COMB(\mathcal{H}_2^{i-1}, \mathcal{N}_2^{i-1}) \rightarrow \mathcal{H}_2^i$, by which, peripheral features ($\mathcal{N}_2^{i-1}$) can incrementally contribute to a central node until all node features are well extracted. In opposite, as a one-pass algorithm, HDC inherently doesn't require storing the history of hypervectors, therefore, to save memory space, existing methods either exclude the features of central nodes from combination, e.g., $BUNDLE(\mathcal{N}_2) \rightarrow \mathcal{H}_2$, or degrade them equally with that of neighbors, e.g., $BUNDLE(\mathcal{N}_2 \oplus H_2) \rightarrow \mathcal{H}_2$. In this work, we propose a simple concatenation method, where, a central node and its peripheral nodes, as a whole, individually occupy half of the hypervector length to highlight central features like standard GNNs.

III. Strengthening the Representation of Hypervectors

A. Plastic Random Bit-Flip Encoding

Value-involved quantization for hypervectors. To overcome the distortion in hypervector encoding, inspired by a previous works [15], we propose an encoding method that utilizes the values of features to proportionally and randomly flip the bits to generate new hypervectors. Initially, the n features of a node constitute n independent hypervector spaces. For the i-th space $\mathcal{L}^i$, all possible values it can sample form a quantization space, which is defined by upper and lower bounds and a sampling distribution. Thus, the range between the upper and lower bounds can be divided into m quantization centers $(\mu_1^i, \mu_2^i, \ldots, \mu_m^i), \mu_x^i < \mu_y^i$ *when* $x < y$, called *clusters*, with each cluster mapped to a representative hypervector l_j^i. Consequently, $\mathcal{L}^i$ can be represented by m ordered hypervectors $\mathcal{L}^i = (l_1^i, l_2^i, \cdots, l_m^i)$ to encode the m quantization centers, that is, when the value of the i-th feature f_i falls into the j-th cluster μ_j^i, f_i is directly encoded as l_j^i. Under this method, the only remaining challenge is how to initialize an ordered set of hypervectors l_j^i to faithfully capture the distribution of feature values.

Plastical orthogonal ring. In order to achieve a quantization hypervector (l_j^i) that's equivalent to numerical space (μ_j^i), a orthogonal ring method is proposed in [7]. It constructs multi-level hypervectors $(l_2^i \cdots, l_{m-1}^i)$ with gradually and randomly flipping between two orthogonal vectors (l_1^i, l_m^i), ensuring the similarity between adjacent vectors is a fixed quantization step (Fig. 3), i.e., $\delta(l_j^i, l_{j+1}^i) = \frac{1}{2(m-1)}$ and $\delta(l_1^i, l_m^i) = 0$ (Note: the zero δ hints that there are half different bits between l_1^i and l_m^i). However, this method still introduces significant noise, which is primarily due to the poor accuracy in those dense regions, where finer granularity must be very useful in quantization. Therefore, an improved *plastical orthogonal ring* is built to allocate different bit-flipping rates (r_i) according to the numerical distance between two plastical clusters, rather than a fixed step, meanwhile ensuring $\delta(l_1^i, l_m^i) \approx 0$ (i.e., *approximately orthogonal*). By introducing a plastic factor $\frac{\mu_{i+1}-\mu_i}{\mu_m-\mu_1}$, each quantization step becomes flexible as:

$$r_i = \frac{1}{2(m-1)} \times \left(\frac{\mu_{i+1} - \mu_i}{\mu_m - \mu_1} + 1 \right) \quad \text{for} \quad 1 \leq i \leq m-1 \qquad (1)$$

where, the former term is the fixed step that divides half of a hypervector ($\frac{1}{2}$) into $m-1$ bit-flipping steps and the later term uses the plastic factor to decide the adaptation to each step (Fig. 3). This adaptation can provide more scalability for those dense regions, at the same time, guarantee the orthogonal ring like [7].

Non-uniform quantization with Hypervectors. Based on the plastical orthogonal ring, the non-uniform quantization in numeral space can be isomorphically mapped into the non-equidistant hypervector space. For example, in Fig. 4, the dense region is quantized as narrower clusters, dropped int which, the corresponding hypervectors also have larger similarity since they have undergone fewer bit flips during initialization. After obtaining the orthogonal hypervectors of all features ($f_i \rightarrow l^i, 1 \leq i \leq n$), those hypervectors are further bundled to generate the representation of a node, i.e., $\mathcal{H}_v = \oplus_{i=1}^n l^i, v \in \mathcal{V}$.

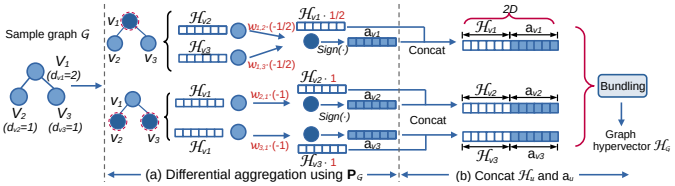

Fig. 5: Aggregation using hyper-weight matrix and equal concatenation.

B. Differential Aggregation for Structural Enhancement

Based on the discussion in Sec. II-C, to be equivalent to GNNs, an HDC-based aggregation should have the ability to 1) estimate the structures around a central node, 2) assess the strength of connections symmetrically between nodes, and 3) inject peripheral nodes into the central node as the aggregation (a_u). Recognizing the disability of the simple binding operation, we propose a novel differential scheme to take charge of the hypervector aggregation. Above all, in a graph, the feature distance between any two nodes can be represented by their similarity of hypervectors, i.e., $w_{u,v} = \delta(\mathcal{H}_u, \mathcal{H}_v), u, v \in \mathcal{V}$, which forms a weight matrix $\mathbf{W}_\mathcal{G} \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$ among nodes to involve all connecting strength. Then, the connectivity $t_{u,v}$ between a central node u and all other nodes, e.g., v, is defined as: 1) $t_{u,v} = \frac{1}{d_u}$ if $u = v$, 2) $t_{u,v} = -\frac{1}{d_u}$ if $e_{u,v} \in \mathcal{E}$, and 3) otherwise $t_{u,v} = 0$, where d_u is the degree of node u. Therefore, all $t_{u,v}$ constitute another transition matrix $\mathbf{T}_\mathcal{G} \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$ that asserts the peripheral structure of each node.

By a Hadamard product ($\circ$) of above two matrices, both of feature similarity and connectivity are merged into a sole ***hyper-weight matrix***, i.e., $\mathbf{P}_\mathcal{G} = \mathbf{W}_\mathcal{G} \circ \mathbf{T}_\mathcal{G}$, where $p_{u,v} = w_{u,v} \times t_{u,v}$. In short, the $\mathbf{P}_\mathcal{G}$ has three explainable properties in below. ***First***, $\mathbf{T}_\mathcal{G}$ structurally filter the similarity values of all elements in $\mathbf{W}_\mathcal{G}$, by which, the resultant matrix $\mathbf{P}_\mathcal{G}$ yields perfect alignment with graph structure. ***Second***, as the basis of u-centered aggregation, the u-th diagonal element $p_{u,u} = \frac{1}{d_u}$ is inversely proportional to the degree of node u, which hints that a central node could contribute less during aggregation when it has more neighbors. ***Third***, the off-diagonal elements in the u-th row, i.e., $p_{u,v} = -\frac{\delta(\mathcal{H}_u,\mathcal{H}_v)}{d_u}$ for $\forall u \neq v$, are the scaled-down similarities, in which, $\frac{1}{d_u}$ ensures an equal contribution of all neighbor as central, meanwhile, the negative sign ("$-$") is used to precisely extract the difference between a central and its environment. Our results in Sec. V-E show that the hyper-weight matrix is powerful in merging structure and similarity.

Aggregation using hyper-weight matrix. Since matrix $\mathbf{P}_\mathcal{G}$ incorporates both structural and distance information, we use it as the weight matrix for graph aggregation, such as a_u of node u, as shown in below:

$$a_u = Sign(\bigoplus_{v\in\mathcal{N}_u^*} (p_{u,v} \times \mathcal{H}_v)) \quad \text{with} \quad \mathcal{N}_u^* = \mathcal{N}_u \cup \{u\} \tag{2}$$

where, $\mathcal{N}_u$ contains all the neighbors of u, the bundling operation ($\oplus$) collects all weighted similarities, both central u and its periphery, as the aggregation on u, and the $Sign$ operation maps each elements to $+1$ or -1 according to the sign of the collected results for a convenient combination in next (Fig. 5(a)). In summary, the similarity and connectivity between centrals and peripheries are concentrated in the hyper-weight matrix to aggregate surrounding hypervectors, which faithfully mimics the embedding flow in traditional GNNs.

C. Concatenation for Lightweight Combination

As mentioned in Sec. II-D, current HDC methods always neglect central nodes, which weakens their bundling-based combination. To address this issue, in this work, we directly concatenate the feature hypervectors with its aggregation, i.e., $\tilde{\mathcal{H}}_u = CONCAT(\mathcal{H}_u, a_u)$, which allows a central to occupy half of its entire hypervector length and significantly enhances its role in the final *reasoning* (Fig. 5(b)). The effectiveness of this method stems from three sources: 1) a_u provides all peripheral situation of a node, including its neighbor similarity and connectivity; 2) as the initial feature, $\mathcal{H}_u$ memorizes the intrinsic properties of a central; and 3) the equal occupancies of the two information balance their impacts successfully. In addition, the ultra-short hypervectors used in our implementation removes the worry about the memory expansion by the twice length with $\tilde{\mathcal{H}}_u$ (Sec. IV-B).

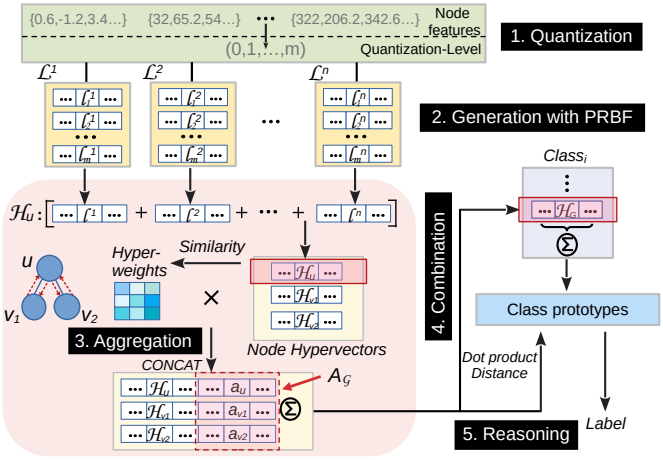

Fig. 6: The framework of CiliaGraph.

IV. Implementation of Memory-Efficient CiliaGraph

A. The Workflow of CiliaGraph

Based on the above operations, CiliaGraph, a new HDC-based graph processing framework, is proposed to achieve both memory-efficient and fast graph classifications, as shown in Fig. 6.

1) Quantization based on prior distribution. To determine the quantization levels mentioned in Sec. III-A, our approach necessitates a priori distribution of each feature that's pertinent to specific tasks, e.g., COIL-RAG, Synthie [10] ... The statistical outcomes are fed into a K-means algorithm [16] to be classified as m quantization centers, i.e., $(\mu_1^i, \mu_2^i, \ldots, \mu_m^i)$, which are associated with their hypervector $\mathcal{L}^i$ for a node i. This method is able to statically find fine-grained centers within sampling ranges, improving the representability compared to traditional uniform levels of quantization [7].

2) Generation of feature-rich hypervectors. To encode n numerical features of a node u into a hypervector, first of all, those features $F_u = (f_1, f_2, \cdots, f_n)$ are individually clustered into quantization centers and replaced with their hypervectors, i.e., $(l^1, l^2, \ldots, l^n)$, by being fetched from $\mathcal{L}^i$. After that, they bundled into a $\mathcal{H}_u$ hypervector, see Sec. III-A. In theory, this approach can merge and encode any number of feature elements without the memory explosion problem under feature-rich contexts.

3) Aggregation using symmetric flows. For symmetric aggregation like GNNs, CiliaGraph propagates hypervectors bidirectionally along all edges, resulting in a central aggregation process at each node. This flow is controlled by the Hyper-weight matrix and all aggregated results are stored in a aggregation matrix $A_\mathcal{G}(a_1, a_2, \cdots, a_{|\mathcal{V}|})$, occupying $|\mathcal{V}| \times D$ memory space. Due to the highly parallelizable nature with our vectorized aggregation, it can easily leverage almost all parallel architectures, such as multi-cores, GPUs and FPGAs.

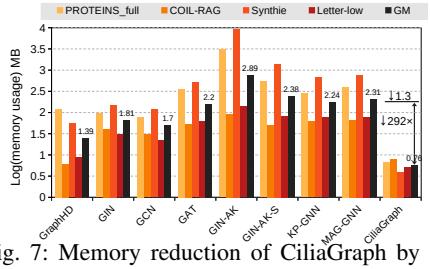

Fig. 7: Memory reduction of CiliaGraph by 1-2 orders compared to the eight SOTAs.

TABLE I: Memory, training time, and test accuracy of seven models. The top three are highlighted by **First**, **Second**, **Third**.

Dataset \ Methods	GraphHD (CPU/GPU)	GIN	GCN	GAT	GIN-AK	GIN-AK-S	KP-GIN	MAG-GNN	**CiliaGraph(Ours)** (CPU/GPU)
Training Time (s)↓									
PROTEINS_full	3.2 / 7.6	17.2	15.5	36.3	58.8	238.2	58.7	60.3	**1.3** / 1.4
COIL-RAG	4.1 / 14.4	42.4	35.6	94.3	135.8	492.3	102.6	120.5	**3.2** / 4.5
Synthie	1.8 / 3.1	8.5	7.4	11.4	65.2	156.7	30.1	38.8	**0.5** / 0.6
Letter-low	5.2 / 30.0	23.6	22.3	29.5	74.5	303.9	60.3	70.6	**1.9**/ 2.6
Test Accuracy (%)↑									
PROTEINS_full	57.84	68.23	70.72	65.55	67.86	65.78	**73.01**	**73.13**	**73.96**
COIL-RAG	6.79	90.75	89.41	91.03	**96.51**	**98.36**	95.01	95.79	**95.88**
Synthie	50.63	63.31	59.50	63.84	**94.15**	92.03	92.31	**94.35**	**93.67**
Letter-low	50.81	94.02	85.66	81.76	**98.47**	96.28	96.44	**97.42**	**97.76**

4) Combination with high memory-efficiency. A concatenation-based combination requires accessing two data in memory: the feature hypervector and its aggregated hypervector. Then, these hypervectors are concatenated together, i.e., $[\mathcal{H}_i, a_i]$, and directly forwarded to subsequent *reasoning* step. By representing l^i and a_i with only a few hundred bits, e.g., $D = 120$ in our experiments (Sec. V-C), CiliaGraph ensures that the concatenation remains memory-efficient.

5) Reasoning for one-shot graph classification. The reasoning is same to the classical HDC processing that bundles all graph nodes together, i.e., $\oplus_{i=1}^{|\mathcal{V}|}[\mathcal{H}_i, a_i]$, and calculates the dot product distances between the result and all class prototypes (a set of label hypervectors). The class with the smallest distance is finally selected as the reasoning output. In next, we present the theoretical evidence about why our short hypervectors outperforms conventional long vectors in memory usage.

B. Memory Efficiency of CiliaGraph: A Theoretical Analysis

The substantial reduction in hypervector length achieved by our CiliaGraph is theoretically grounded in below. For two unit vectors x and y, they are ϵ-*quasi-orthogonal* if $(|x \cdot y| \leq \epsilon)$ and $\epsilon \in [0, 1)$, where ϵ is an angle relaxation easing the strict orthogonality [17]. Intuitively, when using ϵ-*quasi-orthogonal* to replace the traditional strict one in HDC, a larger ϵ can accommodate more mutually orthogonal hypervectors under a fixed vector length D. The theories in [12] and [17] have established that the magnitude of ϵ-*quasi-orthogonal* bipolar hypervectors, i.e., $dim_\epsilon(D)$, grows exponentially with D, i.e., $dim_\epsilon(D) = e^{D\epsilon^2/2}$. Since our *plastical orthogonal ring* (Sec. III-A) only requires the approximate orthogonality (Fig. 3) between the first and last hypervectors in each feature space ($\delta(l_1^i, l_m^i) \approx 0$), it can hold a large amount of features with significantly reduced D. The formal proof is shown in below.

Theorem 1. *For a graph with n node attributes and m quantization levels, using plastic random bit-flip encoding with quasi-orthogonality threshold $\epsilon = \frac{2}{(m-1)}$, the minimum dimension D required to ensure at least $2n$ quasi-orthogonal hypervectors satisfies $D \approx \frac{(m-1)^2}{2} \ln 2n$.*

Proof. The Hamming distance between l_1^i and l_m^i for the i-th space $\mathcal{L}^i$ is $\sum_{i=1}^{m-1} r_i \times D = \frac{D \times m}{2(m-1)}$, indicating the number of differing bits. Converting these into unit vectors, the value of ϵ is obtained as $\epsilon = |l_1^i \cdot l_m^i| = |1 - 2 \times \frac{m}{2(m-1)}| = \frac{1}{(m-1)}$. To ensure robust quasi-orthogonality while reducing computational overhead, $\epsilon = \frac{2}{(m-1)}$ is set. Since l_1^i and l_m^i from each $\mathcal{L}^i$ must maintain quasi-orthogonality, and this property must hold across all n attribute groups, we require $dim_\epsilon(D) = 2n$. Thus, we derive the minimum required dimensions $D \approx \frac{(m-1)^2}{2} \ln 2n$. □

According to above theory, the optimal D for the typical datasets tested in this work are between $[37, 122]$, reducing $1 - 2$ order of memory usage compared to traditional HDC-based GNNs (Sec. V-A).

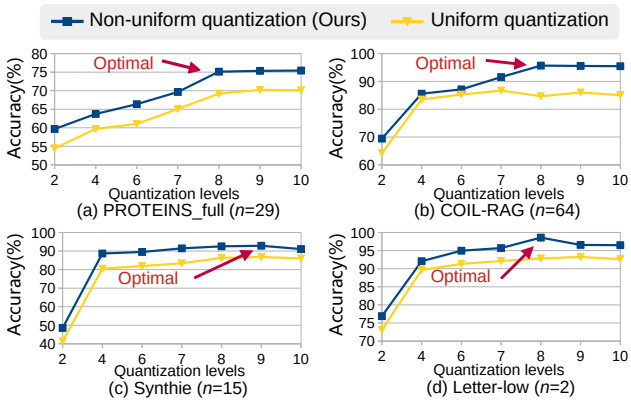

Fig. 8: Optimal quantization levels.

V. Experimental Results

To setup the experiments, seven real-world datasets selected from Tudataset testbed [10] to cover variant magnitudes of node features (n), including Letter-high (2), Letter-low (2), BZR (3), AIDS (4), Synthie (15), PROTEINS_full (29) and COIL-RAG (64).

For the baseline models, we tested: 1) HDC-GNN: GraphHD; 2) GNNs: GIN [18], GCN [19], GAT [20]; 3) SOTA k-hop GNNs: GIN-AK+, GIN-AK+-S [8], KPGIN [13] and MAG-GNN [21], where GIN-AK+-S serves as a sampling model for GIN-AK+ that reduces resource overhead. All GNNs are replicated on GPUs (duel NVIDIA GeForce RTX 4090), while HDCs are tested on both CPUs (AMD EPYC CPUs with 96 cores) and GPUs.

A. Memory reduction of CiliaGraph

As shown in Figure 7, the memory consumption of our proposed CiliaGraph is compared against eight baseline methods across four real-world graph datasets, with the vertical axis representing the logarithmic value of memory usage. By enjoying the ultra-short hypervectors ($D = 120$), CiliaGraph achieves an average memory saving of $292\times$, corresponding to a reduction of $1 - 2$ orders of magnitude in memory consumption. Notably, on the Synthie dataset, CiliaGraph exhibits only a 1.48% performance difference from the best-performing GNN model (Table I) while reducing memory usage by an impressive $2341\times$. This demonstrates that CiliaGraph is a memory-efficient graph classification model, making it well-suited for practical deployment and operation in resource-constrained environments such as on-edge graph processing.

B. Latency and Accuracy of Graph Classification Tasks

The ultra-shot hypervectors of CiliaGraph significantly reduce processing latency across both CPU and GPU platforms for the full graph classification tasks, while maintaining competitive accuracy (Table I). CiliaGraph achieves 85.09% higher accuracy than GraphHD on COIL-RAG, demonstrating its advantage via involving the intrinsic attributes of nodes. Particularly on the Synthie dataset [22], CiliaGraph's performance trails the best GNN model by only 1.48%, while offering a $2341\times$ reduction in memory usage and

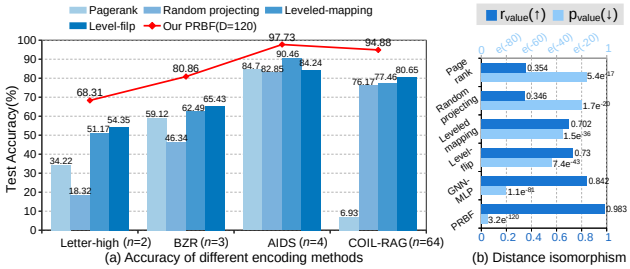

Fig. 9: The impact of different encoding methods.

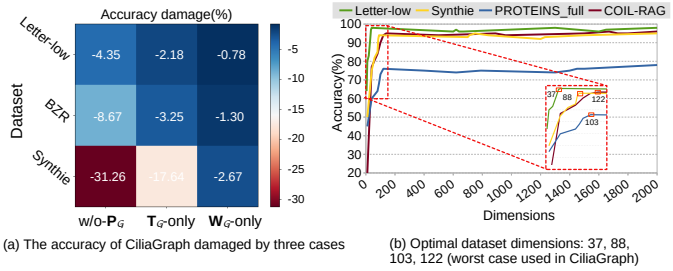

Fig. 10: Hyper-weights ablation and dataset-specified dimensions.

a 313× speedup in training. On the GPU platforms, we didn't observe any acceleration of CiliaGraph on GPUs. The *nvprof* tool reports reveal that the computing reduction of CiliaGraph leads to the negligible *kernel* workloads that are even insufficient to hide the CPU-GPU transferring time for the hypervectors, which shows that CiliaGraph is such an energy-efficient model that general accelerators could be needlessly.

C. Picking Quantization Levels for the Worst Cases

To pick quantization levels can significantly affect data clustering and model accuracy. According to the formula provided in Theorem 1, the quantization level also determines the minimum required dimension. Thus, in all SOTA HDC solution, the dimension $D = 10,000$ is initially set to identify the optimal quantization levels. Fig. 8 illustrates the results for four datasets, with the yellow line indicating uniform quantization. Most datasets achieve the highest and most stable accuracy at $m = 8$. Subsequently, the formula changes to $D \approx \frac{(8-1)^2}{2} \ln 2n$. Therefore, even for the COIL-RAG dataset [10], [23], which has the largest number of node attributes at $n = 64$, the minimum required dimension is $D \approx \frac{(8-1)^2}{2} \ln (2 \times 64) \approx 118.87$ bits. For convenient in setting, D is uniformly approximated to 120, rather than 118.87, to standardize the effectiveness evaluation for all tests on CiliaGraph.

D. The Expression of Rich Features with PRBF Encoding

To assess the expressive power of the CiliaGraph node encoding method, we compare it against four alternative frameworks: 1) *PageRank*; 2) *Random projecting*; 3) *Leveled-mapping*; and 4) *Level-flip* with [15] flipping methods. All these four frameworks use the dimension $D = 10000$. As shown in Fig. 9(a), CiliaGraph achieves optimal performance across all tested datasets, with an average accuracy 15% higher than the *Leveled-mapping* and 39% higher than the *Pagerank*. These significant improvements evidence the effectiveness of our model and encoding methodology. Furthermore, the Pearson correlation coefficient [14] is employed to validate that CiliaGraph effectively maintains isomorphism (Fig. 9(b)) in feature distances between nodes before and after encoding, thereby preserving the original data features in $\mathcal{HD}$ space. In addition, the precise encoding is seen as a key reason to combat the Noise-1.

E. Ablation Experiment of Hyper-Weight Matrix

To validate the effectiveness of the more precise similarity weights and transition matrix introduced in Sec. III-B, CiliaGraph is compared to three ablating cases: 1) "w/o-$\mathbf{P}_\mathcal{G}$" is *without similarity weights matirx and transition matrix*; 2) "$\mathbf{T}_\mathcal{G}$-only" is *without similarity weights matrix*; and 3) "$\mathbf{W}_\mathcal{G}$-only" is *without transition matrix*. The accuracy differences of these frameworks are shown in Fig. 10(a). It is observed that the performance of all three frameworks progressively deteriorates, particularly for w/o-$\mathbf{P}_\mathcal{G}$ and $\mathbf{T}_\mathcal{G}$-only. This accentuates the inadequacies of basic edge encoding techniques which overlook the significance of connection strengths. The results of $\mathbf{W}_\mathcal{G}$-only demonstrate that integrating transition matrices with similarity weight matrices can enhance the model expressiveness, offering a refined depiction of inter-node connections. The results demonstrate that only the combined usage of $\mathbf{W}_\mathcal{G}$ and $\mathbf{T}_\mathcal{G}$ can prevent significant Noise-2.

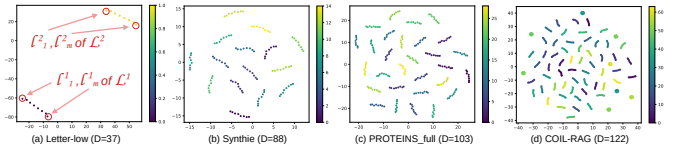

Fig. 11: t-SNE visualisation of initial $\mathcal{L}^i$. Both of coherence and orthogonality are stable across all optimal length of hypervectors (D).

F. The Optimal Dimensions for Individual Datasets

To further quantify the confidence of the minimal D proposed in Sec. IV-B, we first calculate the theoretical values of D via the Theorem 1 for each of datasets, including Letter-low (33.96), Synthie (83.33), PROTEINS_full (99.48), and COIL-RAG (118.87). Then, in measuring part, our experiments span D from the theoretical minimum 5 to 2000 and evaluate the CiliaGraph for the four datasets, as shown in Fig. 10(b). The accuracy curves indicate that they achieve their optimal performances at 37, 88, 103, and 122 respectively. The comparison between the theoretical and measured values hints that our theorem achieves predicting accuracy of 95.81% on average, which is a well predictor of hypervector length. In addition, a dimension reduction of features (t-SNE) in Fig. 11 shows that the generated initial hypervectors exhibit not only intra-cluster coherence (high k-means inertia) but also inter-group quasi-orthogonality among different features, which is the reliable short encoding exactly as expected in Sec. III-A.

VI. Conclusions

This paper presents a novel approach to address the memory challenges posed by long hypervectors in HDC-based graph processing. By introducing three new operators for feature-rich hypervectors, we improve the efficiency of feature representation and extraction, enabling HDC to operate more like traditional GNNs with significantly reduced memory requirements. The experiments indicate that the short vector length we predict is very close to the optimal in terms of minimizing memory usage.

Acknowledgment

This work was supported in part by the National Natural Science Foundation of China under Grant 62272393 and the Aeronautical Science Foundation of China under Grant 20240043053002.

REFERENCES

[1] E. Hassan, Y. Halawani, B. Mohammad, and H. Saleh, "Hyper-dimensional computing challenges and opportunities for ai applications," *IEEE Access*, vol. 10, pp. 97 651–97 664, 2021.

[2] J. Morris, K. Ergun, B. Khaleghi, M. Imani, B. Aksanli, and T. Simunic, "Hydrea: Utilizing hyperdimensional computing for a more robust and efficient machine learning system," *ACM Transactions on Embedded Computing Systems*, vol. 21, no. 6, pp. 1–25, 2022.

[3] I. Nunes, M. Heddes, T. Givargis, A. Nicolau, and A. Veidenbaum, "Graphhd: Efficient graph classification using hyperdimensional computing," in *DATE(2022)*, pp. 1485–1490.

[4] H. Li, F. Liu, Y. Chen, and L. Jiang, "Hypernode: An efficient node classification framework using hyperdimensional computing," in *ICCAD(2023)*, pp. 1–9.

[5] J. Kang, M. Zhou, A. Bhansali, W. Xu, A. Thomas, and T. Rosing, "Relhd: A graph-based learning on fefet with hyperdimensional computing," in *ICCD(2022)*, pp. 553–560.

[6] P. Kanerva, "Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors," *Cognitive Computation*, vol. 1, pp. 139–159, 2009.

[7] L. Ge and K. K. Parhi, "Classification using hyperdimensional computing: A review," *IEEE CASS Magazine*, vol. 20, no. 2, pp. 30–47, 2020.

[8] L. Zhao, W. Jin, L. Akoglu, and N. Shah, "From stars to subgraphs: Uplifting any GNN with local structure awareness," in *ICLR(2022)*.

[9] D. Kleyko, D. A. Rachkovskij, E. Osipov, and A. Rahimi, "A survey on hyperdimensional computing aka vector symbolic architectures, part i: Models and data transformations," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–40, 2022.

[10] C. Morris, N. M. Kriege, F. Bause, K. Kersting, P. Mutzel, and M. Neumann, "Tudataset: A collection of benchmark datasets for learning with graphs," *ICML 2020 workshop "Graph Representation Learning and Beyond"*, 2020.

[11] T. Yu, Y. Zhang, Z. Zhang, and C. M. De Sa, "Understanding hyperdimensional computing for parallel single-pass learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 1157–1169, 2022.

[12] Z. Yan, S. Wang, K. Tang, and W.-F. Wong, *Efficient Hyperdimensional Computing*. Springer Nature Switzerland, 2023, p. 141–155.

[13] J. Feng, Y. Chen, F. Li, A. Sarkar, and M. Zhang, "How powerful are k-hop message passing graph neural networks," *Advances in NIPS*, vol. 35, pp. 4776–4790, 2022.

[14] Z. Šverko, M. Vrankić, S. Vlahinić, and P. Rogelj, "Complex pearson correlation coefficient for eeg connectivity analysis," *Sensors*, vol. 22, no. 4, p. 1477, 2022.

[15] I. Nunes, M. Heddes, T. Givargis, and A. Nicolau, "An extension to basis-hypervectors for learning from circular data in hyperdimensional computing," in *DAC(2023)*, pp. 1–6.

[16] E. Umargono, J. E. Suseno, and S. V. Gunawan, "K-means clustering optimization using the elbow method and early centroid determination based on mean and median formula," *Proceedings of the 2nd ISSTEC*, pp. 234–240, 2020.

[17] P. C. Kainen and V. Kůrková, "Quasiorthogonal dimension," in *Beyond traditional probabilistic data processing techniques: Interval, fuzzy etc. Methods and their applications*. Springer, 2020, pp. 615–629.

[18] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *ICLR*, 2019.

[19] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *International Conference on Learning Representations (ICLR)*, pp. 1–14, 2017.

[20] Y. B. Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, "Graph Attention Networks," *ICLR*, vol. 11731 LNCS, pp. 566–577, 2018.

[21] L. Kong, J. Feng, H. Liu, D. Tao, Y. Chen, and M. Zhang, "Mag-gnn: reinforcement learning boosted graph neural network," in *NIPS(2023)*.

[22] A. Feragen, N. Kasenburg, J. Petersen, M. de Bruijne, and K. Borgwardt, "Scalable kernels for graphs with continuous attributes," *Advances in neural information processing systems*, vol. 26, 2013.

[23] K. Riesen and H. Bunke, "Iam graph database repository for graph based pattern recognition and machine learning," in *Structural, Syntactic, and Statistical Pattern Recognition December 4-6, 2008. Proceedings*. Springer, 2008, pp. 287–297.